

Name: Sai Prasanna Cheedi

Student Number: 2407934

Land Cover Detection using Aerial Imagery

1. Introduction

Image classification involves assigning a label or category to an image based on its content. Its applications are spread widely in fields like healthcare, autonomous vehicles and even in everyday tools like facial recognition systems [5]. For traditional approaches it requires manually designing features from the image, which is time-consuming and leads to limitations. With its ability to learn features automatically from raw pixel, Deep learning has reduced such difficulties [1].

This report explores the three approaches used for image classification. A basic Support Vector Machine (SVM) with raw pixel features for classification, HoG (Histogram of Oriented Gradients) feature extraction along with PCA (Principal Component Analysis) which augmented with SVM for preprocessing of images [2, 3] and a CNN (Convolutional Neural Network) for end-to-end learning [4].

SVM model achieved moderate classification accuracy, emphasizing the limitation of working directly with high dimensional data. This is rectified by using hyperparameter tuning and preprocessing. In model 2 an improvement is demonstrated in SVM's classification accuracy by integrating HoG with PCA [3]. CNN achieved highest classification accuracy in all the three sets of data which demonstrates the ability of CNN to generalize well across all image classes [6].

2. Method

2.1 Dataset Preparation and Preprocessing

The dataset with 21 land cover categories from the UC Merced Land along with their corresponding class labels are used in this project. First the data is shuffled and split into 80% training and 20% temporary data and then this 20% of temporary data is divided into 10% validation and 10% test data [7]. The splitting strategy often involves first creating a training set and a temporary holdout set, which is later divided into validation and test sets [5]. "If you do not have a separate test set, you risk selecting a model that performs well on the validation data but does not generalize well to new data" by Geron, A. (2019). The pixel values were normalized to range (0, 1) by dividing with 255 for CNN-based methods to improve training stability [4, 7].

2.2 SVM

We have chosen 5 specific classes on which this whole model is focused on, filtered training data and labels of these classes based on their indices. This is done using `np.where` and `np.isin` [8]. The data sets are in 4D array form with "samples, height, width, channels", to train them with SVM model they are reshaped by flattening into 2D array form with "samples, features" [2, 7].

To balance overfitting and underfitting, a grid search was conducted along with regularization parameter and kernel type of "linear" and "rbf" over hyperparameters. To select best parameters, 3 folds of cross-validation was used [2,7].

SVM model is trained with these parameters and the performance was evaluated on the training, validation and test datasets. Results were summarized as accuracy metrics and confusion matrices.

2.3 HoG Feature Extraction+PCA (SVM model)

This model explains how the features are extracted using HoG feature extraction method and Usage of PCA method for dimensionality reduction. HoG was applied (cell size:8x8 pixels, Block size:2x2 cells, Gradient Orientation number:8) to extract texture and edge features from images [3, 9]. It provides a robust representation of the images, emphasizing gradients and spatial patterns.

PCA model is applied to reduce the dimensionality of HoG features extracted from all classes. It transforms the features space by selecting the top 50 principal components, preserving most of the variance while reducing noise and computational complexity [2, 7]. This results in reduction of computation time and mitigated risk of overfitting.

A linear and rbf kernel SVM was tuned using grid search with 5-fold cross-validation [7]. Based on validation accuracy, hyperparameters were selected and evaluated on the training, validation and test sets.

2.4 CNN

To learn hierarchical features automatically from image data, Convolutional Neural Network (CNN) was built. The construction of the model contains [5, 10]:

- Conv2D: Convolutional layers with filters and kernels for feature extraction.
- MaxPooling2D: Pooling layers for spatial dimensionality reduction.
- Flatten: 2D feature maps are converted into 1D vector for the dense layer.
- Dense and Dropout: Fully connected layers for classification, with dropout to prevent overfitting.
- Dense (21, activation= 'softmax'): Final output layer for multi-class classification with 21 classes.

Model is compiled using Adam optimizer with a learning rate of 0.001, sparse categorical cross-entropy loss where labels are provided as integers and accuracy which is a metric that provides an easy-to-understand performance indicator [1, 5, 10]. To prevent overfitting Early stopping is used. Training of the model ran for 20 epochs with a batch size of 32, leveraging validation data for performance monitoring.

3. Results

Performance of all the 3 models is Visualized in the form of Accuracy metric of training, validation and testing from which the ability of generalization and learning process of the model is known. Confusion matrix is used on test data to show how the model has predicted the classes.

An accuracy curve is resulted for the CNN model between Training and validation. It is not visualized in other two models as their curves are always visualized as straight line because SVM's are not iterative learners and don't have a concept of multiple epochs during training. The features are static and once if they are computed, they don't change.

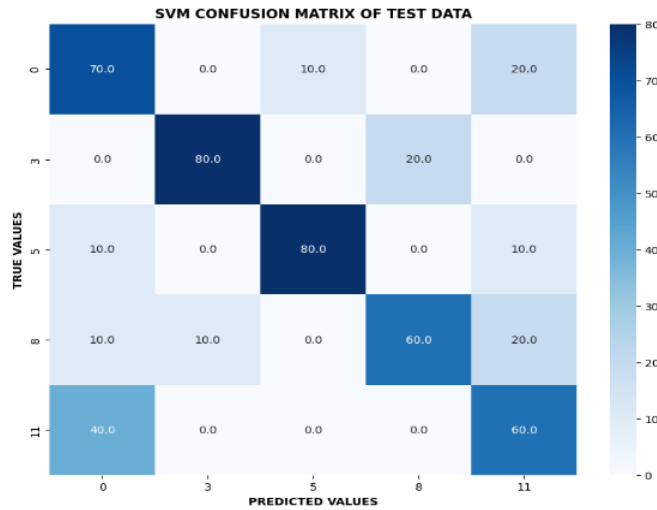
3.1 Model 1

3.1.1 Accuracy percentage

Metric	Training	Validation	Testing
Accuracy	99.5%	74%	70%

A high value of accuracy in training data represents that model has learned patterns in a reasonable way. But slight decrease in validation and testing accuracy indicates overfitting which means the model struggles to generalize to unseen data [4, 5, 7].

3.1.2 Confusion matrix



The diagonal entries represent the percentage of test samples correctly predicted for each class. Test samples which are misclassified into other classes are represented by off-diagonal values. Overall performance of the class was well on class 3 and 5 with correct prediction of 80% [2, 7].

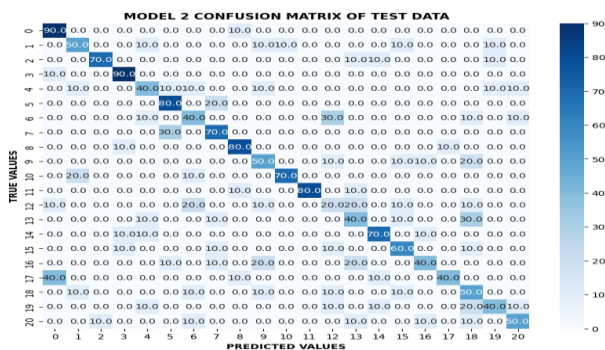
3.2 Model 2

3.2.1 Accuracy percentage

Metric	Training	Validation	Testing
Accuracy	99.28%	55.71%	57.61%

High training accuracy indicates that model captured most of the patterns in training data and fit almost perfectly, but it often raises concerns about overfitting. Large decline is seen in validation accuracy which means model may not generalized well to new data, because of overfitting or insufficient representative towards training data. Test accuracy suggests that model's ability to generalize to unseen data is limited [4, 5, 7].

3.2.2 Confusion matrix



Model is performed well on certain classes where features are clear and separable. Moderate levels of confusion is seen for some classes like 5, 6, 10, 14 which require analysis focus. Classes like 16, 17, 19, 20 highlight week discriminative power, improvement on feature extraction, class balancing [2, 7].

3.3 Model 3

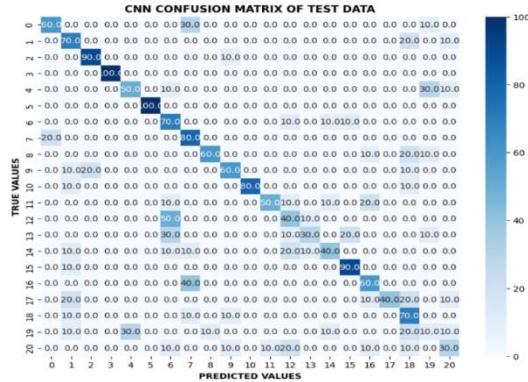
3.3.1 Accuracy percentage

Metric	Training	Validation	Testing
Accuracy	91.67%	60.95%	60.95%

Training accuracy is high which means model has learned all the patterns and perform well on the data it has seen but the ability to generalize to unseen data is limited. Model struggles to generalize beyond the training data as there is a significant drop in validation accuracy. Testing and validation

accuracy are identical which means generalization capability has stabilized but for certain limit [4, 5, 7].

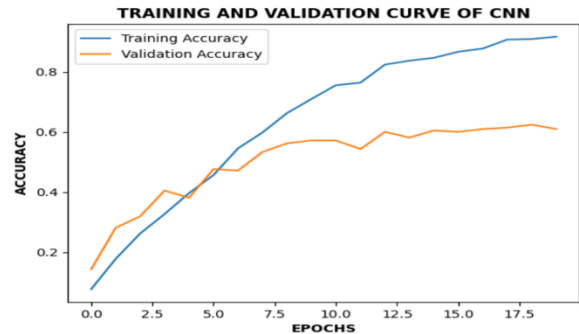
3.3.2 Confusion Matrix



Strong performance of the model is seen on several classes but confusion in some reducing its overall accuracy. Off-diagonal values suggest overlapping patterns or insufficient power of discrimination in certain class features [2, 7]

3.3.3 Training and Validation curves

Training curve visualizes the continuous increase in accuracy reaching more than 90% but there is a significant gap between training and validation. This states the model overfits the training data after epoch 10 by limiting capability of generalization [7].



The process has been executed with different number of folds cross-validation(cv) and different regularization parameters (c) to observe the best performance of the model (balancing underfitting and overfitting). Due to high-dimensionality of the data, performance has always resulted as overfitting.

Above mentioned results are the best one picked from the performances of the model with low rate of overfitting among all the observed or analyzed performances.

4. Conclusion

Simplicity and efficiency nature of linear SVM made ideal for smaller dataset. Moreover, due to its inability to capture spatial relationships it faced problems with high-dimensional data. Feature extraction along with dimensionality reduction as provided moderated performance but risks of information loss and difficulty in differentiating between similar classes. CNN performed very well by learning hierarchical features directly from data. Rather than its computational demands, highest accuracy is delivered making it ideal for large scale, diverse datasets [6].

By applying transformations like rotations and flips, data diversity and generalization could have been improved. Interpretability and robustness might have been enhanced by combining CNN for feature extraction with SVM classifiers.

5. References

1. Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.
https://books.google.com/books/about/Deep_Learning_with_Python_Second_Edition.html?id=XHpKEAAAQBAJ
2. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
<https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>
3. Dalal, N., & Triggs, B. (2005). Histograms of Oriented Gradients for Human Detection. CVPR, 886–893.
<https://ieeexplore.ieee.org/document/1467360>
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
<https://www.deeplearningbook.org/>
5. Geron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
https://books.google.com/books/about/Hands_On_Machine_Learning_with_Scikit_Lea.html?id=F7v0DwAAQBAJ
6. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR.
<https://ieeexplore.ieee.org/document/7780459>
7. Scikit-learn Documentation
Scikit-learn: Machine Learning in Python Documentation.
<https://scikit-learn.org/stable/index.html>
8. NumPy Documentation
NumPy: The fundamental package for array computing with Python.
<https://numpy.org/doc/stable/>
9. Scikit-image Documentation: Histogram of Oriented Gradients (HoG)
Scikit-image: Feature Extraction using Histogram of Oriented Gradients.
https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_hog.html
10. Keras Team. (n.d.). *Keras layers API*.
<https://keras.io/api/layers/>